🧭 Guía Docente – React: Fundamentos y Primer Componente (4 horas)

🎯 Objetivos de la sesión

Al finalizar esta sesión, los estudiantes serán capaces de:

1. Comprender qué es React y por qué es tan usado en el desarrollo moderno.
2. Crear su primer proyecto React con **Vite**.
3. Entender la estructura básica de un proyecto React.
4. Escribir código JSX correctamente y aplicar sus reglas principales.
5. Crear su primer **componente funcional** en React.

🕐 HORA 1 – Introducción, Setup y Conceptos Básicos (60 min)

🧩 1.1 ¿Qué es React? (10 min)

Explicación base:

- React es una **biblioteca de JavaScript** (**no** un framework) creada por Facebook en 2013.
- Sirve para construir **interfaces de usuario** (UI) basadas en **componentes reutilizables**.

Puntos clave:

Concepto	Descripción	Ejemplo visual
Componentes	Bloques de construcción reutilizables.	En Netflix: cada tarjeta de película es un componente.
Declarativo	Tú dices *qué* quieres mostrar, no *cómo* hacerlo.	`<h1>Hola</h1>` en lugar de `document.createElement(...)`
Virtual DOM	React detecta qué cambió y actualiza solo eso, no toda la página.	Cambiar solo un “like” sin recargar todo.

Analogía para la clase:

🧱 “React es como un Lego: cada pieza (componente) puede armar interfaces completas.”

Motivar:

📈 “React es hoy una de las habilidades más demandadas por las empresas. Está en más del 60 % de las ofertas de front-end.”

⚙️ 1.2 Prerrequisitos y entorno (10 min)

Objetivo: Asegurarse de que todos tengan el entorno listo.

Verificar:

- Node.js ≥ 18
- npm ≥ 9
- VS Code instalado

Comandos:

node --version
npm --version

Si falta, ingresar a https://nodejs.org y descargar la versión **LTS**.

Extensiones útiles para VS Code:

- **ES7+ React Snippets** – autocompletado.
- **Prettier** – formateo automático.
- **Simple React Snippets** – plantillas de componentes.

👉 Muestra cómo instalar una extensión paso a paso (pestaña “Extensiones” → buscar → instalar).

⚡ 1.3 Crear Proyecto con Vite (15 min)

Qué explicar:

- **Vite** = herramienta moderna para **crear proyectos frontend** (rápida, ligera y con servidor integrado).
- Sustituye a `create-react-app`.
- Vite es una herramienta que te ayuda a **iniciar, desarrollar y compilar** proyectos de frontend mucho más rápido que herramientas tradicionales como Webpack o Create React App.

Demostración guiada:

#Crear el proyecto
npm create vite@latest project-name

- Framework: **React**
- Variant: **JavaScript**

Luego:

cd mi-primera-app-react #Ingresar a la carpeta del proyecto
npm install #Instalar dependencias
npm run dev #Iniciar el servidor de desarrollo

🔎 **Objetivo de este paso:** Que todos vean React corriendo en su navegador (`http://localhost:5173`).

Momento clave:

🎉 "¡Acaban de crear su primera aplicación React!"

🗂️ 1.4 Estructura del proyecto (15 min)

Abrir VS Code y revisar carpetas:

```
mi-primera-app-react/
├── node_modules/        # ⚠️ NO tocar - librerías instaladas
├── public/              # Archivos estáticos (imágenes, etc)
│   └── vite.svg
├── src/                 # 👈 AQUÍ trabajaremos
│   ├── assets/          # Imágenes, fonts, etc
│   ├── App.css          # Estilos del componente App
│   ├── App.jsx          # 🔴 Componente principal
│   ├── index.css        # Estilos globales
│   └── main.jsx         # 🔴 Punto de entrada
├── .gitignore
├── index.html           # 🔴 HTML principal
├── package.json         # Configuración y dependencias
└── vite.config.js       # Configuración de Vite
```

Carpeta/Archivo	Descripción
public/	Archivos estáticos (imágenes, íconos).
src/	Código fuente (aquí trabajaremos).

App.jsx	Componente principal.
main.jsx	Punto de entrada donde React se “monta” al HTML.
index.html	El HTML base (solo tiene un `<div id="root">`).
eslint.config.js	Configuración de ESLint, una herramienta que ayuda a detectar y corregir errores de código o malas prácticas en JavaScript/React.

Explicar:

- `createRoot(document.getElementById("root"))` → “Monta” la app React en el div `root`.
- Todo lo que se renderiza en React aparece dentro de ese div.

💡 **Analogía:** “El `index.html` es como el escenario; React coloca allí los actores (componentes).”

Archivos más importantes:

1. `index.html` (raíz del proyecto):

```
<!doctype html><html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Vite + React</title>
  </head>
  <body>
    <div id="root"></div>
    <!-- Contenedor principal donde React renderiza la aplicación -->

    <script type="module" src="/src/main.jsx"></script>
    <!-- Importa el archivo principal de JavaScript (React) como módulo ES -->

  </body>
</html>
```

2. `src/main.jsx` (punto de entrada):

```
import { StrictMode } from 'react'
// Importa StrictMode de React para ayudar a identificar problemas potenciales en la aplicación.
```

```
import { createRoot } from 'react-dom/client'
// Importa createRoot desde react-dom/client para inicializar el renderizado con la nueva API de React 18.

import './index.css'
// Importa los estilos globales definidos en index.css.

import App from './App.jsx'
// Importa el componente principal de la aplicación desde App.jsx.

createRoot(document.getElementById('root')).render(
  // Crea el punto de entrada de React en el elemento con id 'root'
  // y renderiza la aplicación.

  <StrictMode>
    <App />
  </StrictMode>,
  // Renderiza el componente App envuelto en
  //StrictMode para activar comprobaciones adicionales en desarrollo.
)
```

🧹 1.5 Limpiar el proyecto (10 min)

Objetivo: Eliminar el código de ejemplo de Vite para empezar desde cero.

- Limpiar `App.jsx`, `App.css` e `index.css`.
- Escribir un componente simple:

```
function App() {
  return (
    <div>
      <h1>Mi Primera App en React</h1>
      <p>¡Hola mundo!</p>
    </div>
  )
}

export default App
```

Resultado esperado: Ver “Mi Primera App en React” en el navegador.

🕑 HORA 2 – JSX y Reglas Fundamentales (60 min)

⚛️ 2.1 ¿Qué es JSX? (15 min)

Definición:

- JSX = *JavaScript XML*
- Permite escribir código similar a HTML dentro de JS.

const elemento = <h1>Hola Mundo</h1>;

👉 React convierte JSX a `React.createElement(...)` automáticamente.

Beneficio:

Es **más legible**, más **rápido de escribir** y permite **combinar lógica y UI** en un solo archivo.

📜 2.2 Reglas fundamentales de JSX (25 min)

Regla 1 – Un solo elemento raíz

- Todo componente debe devolver **un único contenedor** (`div` o `<> </>`).
- **Fragmentos (`<>`)** permiten agrupar sin crear etiquetas adicionales.

Regla 2 – Cerrar etiquetas

- `<img />`, `<input />`, `<br />` → siempre cerradas.

Regla 3 – Usar `className`

JSX no permite `class` (palabra reservada JS).

-

Regla 4 – Atributos en camelCase

HTML	JSX
onclick	onClick
maxlength	maxLength

Regla 5 – Interpolar JavaScript con `{}`

Permite usar variables y expresiones:

```
const nombre = "Laura"
<p>Hola, {nombre}</p>
```

También se pueden usar operaciones:

```
{edad >= 18 ? 'Mayor de edad' : 'Menor de edad'}
```

Regla 6 – Comentarios

```
{/* Esto es un comentario JSX */}
```

💡 **Consejo didáctico:**

Muestra errores comunes y haz que los corrijan en vivo (por ejemplo, dejar dos etiquetas raíz o no cerrar `<img>`).

💻 2.3 Ejercicio práctico – Tarjeta Personal (20 min)

Objetivo: Aplicar las reglas de JSX y practicar variables y expresiones.

Código base:

```
import './App.css'

function App() {
  const nombre = "Jhan Carlos Martinez"
  const profesion = "Ingeniero de Sistemas"
  const edad = 24
  const ciudad = "Armenia Quindio"
  const habilidades = ["React", "JavaScript", "CSS"]
  const disponible = true

  return (
    <div>
      <h1>Perfil Profesional</h1>
      <h2>{nombre}</h2>
      <p><strong>Profesión:</strong> {profesion}</p>
      <p><strong>Edad:</strong> {edad} años</p>
      <p><strong>Ciudad:</strong> {ciudad}</p>
      <p><strong>Estado:</strong> {disponible ? '✅ Disponible' : '❌ No disponible'}</p>

      <h3>Habilidades:</h3>
      <p>{habilidades.join(', ')}</p>
    </div>
```

```
  )
}

export default App

/* App.css */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: Arial, sans-serif;
}

/* Estilos para el contenedor principal */
div {
  max-width: 600px;
  margin: 40px auto;
  padding: 30px;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  border-radius: 15px;
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.2);
  color: white;
}

/* Título principal */
h1 {
  text-align: center;
  font-size: 2.5em;
  margin-bottom: 10px;
  text-transform: uppercase;
  letter-spacing: 2px;
  text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.3);
}

/* Nombre */
h2 {
  text-align: center;
  font-size: 2em;
  margin: 20px 0;
  padding-bottom: 15px;
  border-bottom: 3px solid rgba(255, 255, 255, 0.3);
}

/* Subtítulo de habilidades */
```

```
h3 {
  font-size: 1.5em;
  margin-top: 25px;
  margin-bottom: 10px;
  color: #ffd700;
  text-shadow: 1px 1px 2px rgba(0, 0, 0, 0.2);
}

/* Párrafos */
p {
  font-size: 1.1em;
  line-height: 1.8;
  margin: 12px 0;
  padding-left: 10px;
}

/* Texto en negrita */
strong {
  color: #ffd700;
  font-weight: 600;
}
```

Tareas para el grupo:

1. Cambiar los valores por su propia información.
2. Agregar un campo `email`.
3. Calcular años de experiencia (ej: `edad - 22`).
4. Cambiar `disponible` a `false` para ver el resultado condicional

SESIÓN 2

HORA 1: Componentes y Props (60 min)

1.1 ¿Qué es un Componente? (10 min)

Definición: Un componente es una pieza independiente y reutilizable de la UI.

Los componentes permiten modularizar el código, facilitando su mantenimiento y reutilización.

Analogía:

- Tu app es como una casa

- Los componentes son como ladrillos LEGO
- Puedes reutilizar el mismo "ladrillo" en diferentes partes

Tipos de componentes:

- **Funcionales** (los que usaremos): funciones que retornan JSX
- ~~Clase~~ (obsoletos, no los enseñaremos)

Anatomía de un componente funcional:

```
function NombreDelComponente() {
  // Lógica del componente (opcional)

  return (
    // JSX que se renderizará
  )
}

export default NombreDelComponente
```

Reglas para nombrar componentes:

- ✅ SIEMPRE en PascalCase: `MiComponente`, `Header`, `UserCard`
- ❌ NO en camelCase: `miComponente`
- ❌ NO en minúsculas: `header`

1.2 Crear el Primer Componente (15 min)

Crear un componente Header:

PASO 1: Crear carpeta `components` dentro de `src`

```
src/
├── components/    # 👈 Nueva carpeta
├── App.jsx
└── ...
```

PASO 2: Crear archivo `src/components/Header.jsx`

```
// Header.jsx
function Header() {
  return (
    <header>
      <h1>🚀 Mi Aplicación React</h1>
```

```
    <p>Bienvenido a mi primera app</p>
  </header>
 )
}

export default Header
```

Explicar:

- `function Header()`: define el componente
- `return (...)`: retorna el JSX
- `export default Header`: permite importarlo en otros archivos

PASO 3: Importar y usar en `App.jsx`

```
// App.jsx
import Header from './components/Header'

function App() {
 return (
  <div>
   <Header />

   <main>
    <p>Contenido principal de la aplicación</p>
   </main>
  </div>
 )
}

export default App
```

Explicar:

- `import Header from './components/Header'`: importa el componente
- `<Header />`: usa el componente (como si fuera una etiqueta HTML)
- `./` significa "en la misma carpeta o subcarpeta"

Guardar y verificar en el navegador.

1.3 Props: Pasar Datos a Componentes (20 min)

¿Qué son las Props?

- **Props = Properties** (propiedades)
- son parámetros que se pasan de un componente padre a uno hijo.
- Permiten pasar datos del componente padre al hijo
- Son como los argumentos de una función

Ejemplo básico:

Crear `src/components/Saludo.jsx`:

```
// Saludo.jsx
function Saludo(props) {
  return (
    <div>
      <h2>Hola, {props.nombre}!</h2>
      <p>Tienes {props.edad} años</p>
    </div>
  )
}

export default Saludo
```

Usar en `App.jsx`:

```
import Header from './components/Header'
import Saludo from './components/Saludo'

function App() {
  return (
    <div>
      <Header />

      <Saludo nombre="Carlos" edad={30} />
      <Saludo nombre="Ana" edad={25} />
      <Saludo nombre="Luis" edad={35} />
    </div>
  )
}

export default App
```

Explicar:

- `nombre="Carlos"`: prop de tipo string (entre comillas)

- `edad={30}`: prop de tipo number (entre llaves)
- El mismo componente `Saludo` se reutiliza 3 veces con datos diferentes

---

Destructuring de Props (sintaxis moderna):

En lugar de escribir `props.nombre`, `props.edad`... podemos desestructurar:

```
// Saludo.jsx - FORMA MEJORADA
function Saludo({ nombre, edad }) {  // 👈 Destructuring
  return (
    <div>
      <h2>Hola, {nombre}!</h2>
      <p>Tienes {edad} años</p>
    </div>
  )
}

export default Saludo
```

Más limpio y legible. ¡Usaremos siempre esta forma!

Props de diferentes tipos:

```
// Ejemplos de props
<Componente
  texto="Hola"          // String
  numero={42}           // Number
  booleano={true}       // Boolean
  array={[1, 2, 3]}     // Array
  objeto={{ x: 10 }}    // Object
  funcion={() => {}}    // Function
/>
```

Recuerda: Todo menos strings va entre llaves `{}`

1.4 Ejercicio: Componente Card (15 min)

Crear un componente reutilizable de tarjeta de producto.

Crear `src/components/ProductCard.jsx`:

```
// ProductCard.jsx
function ProductCard({ nombre, precio, imagen, disponible }) {
  return (
    <div style={{
      border: '1px solid #ddd',
      padding: '20px',
      margin: '10px',
      borderRadius: '8px',
      width: '250px'
    }}>
      <img
        src={imagen}
        alt={nombre}
        style={{ width: '100%', height: '200px', objectFit: 'cover' }}
      />
      <h3>{nombre}</h3>
      <p style={{ fontSize: '20px', color: 'green', fontWeight: 'bold' }}>
        ${precio}
      </p>
      <p style={{ color: disponible ? 'green' : 'red' }}>
        {disponible ? '✅ En stock' : '❌ Agotado'}
      </p>
    </div>
  )
}

export default ProductCard
```

Usar en `App.jsx`:

```
import Header from './components/Header'
import ProductCard from './components/ProductCard'

function App() {
  return (
    <div>
      <Header />

      <div style={{ display: 'flex', flexWrap: 'wrap' }}>
        <ProductCard
          nombre="Laptop HP"
          precio={1200}
          imagen="<https://via.placeholder.com/250>"
          disponible={true}
```

```
      />

      <ProductCard
        nombre="Mouse Logitech"
        precio={25}
        imagen="<https://via.placeholder.com/250>"
        disponible={false}
      />

      <ProductCard
        nombre="Teclado Mecánico"
        precio={80}
        imagen="<https://via.placeholder.com/250>"
        disponible={true}
      />
    </div>
  </div>
 )
}

export default App
```

Pedir a los estudiantes:

1. Agregar 2 productos más
2. Cambiar las imágenes (pueden usar https://picsum.photos/250 para imágenes random)
3. Modificar los estilos inline

CardV2

ProductCard2.jsx

```
import './ProductCard.css'

function ProductCard2({ imagen, nombre, precio, disponible }) {
 return (
  <div className="producto-card">
   <img src={imagen} alt={nombre} className="producto-imagen" />
   <h3>{nombre}</h3>
   <p className="producto-precio">${precio}</p>
   <p className={`producto-stock ${disponible ? 'disponible' : 'agotado'}`}>
    {disponible ? '✅ En stock' : '❌ Agotado'}
   </p>
  </div>
 )
```

```
}

export default ProductCard2
```

ProductCard.css

```
.producto-card {
  border: 1px solid #ddd;
  padding: 20px;
  margin: 10px;
  border-radius: 8px;
  width: 250px;
}

.producto-imagen {
  width: 100%;
  height: 200px;
  object-fit: cover;
}

.producto-precio {
  font-size: 20px;
  color: green;
  font-weight: bold;
}

.producto-stock.disponible {
  color: green;
}

.producto-stock.agotado {
  color: red;
}
```

HORA 2: Estilos en React (60 min)

2.1 Formas de Aplicar Estilos (10 min)

En React hay 3 formas principales:

1. **Estilos inline** (objetos JavaScript)
    1. útil para estilos dinámicos pero difícil de mantener.
2. **CSS externo** (archivos .css separados)

        1. Común y fácil de usar, pero puede haber conflictos de nombres.
    3. **CSS Modules** (CSS con scope local)
        1. previene conflictos de clases, ideal para aplicaciones grandes.

Hoy veremos las 3 formas.

2.2 Estilos Inline (15 min)

Sintaxis:

```
<div style={{ propiedad: 'valor' }}>
  Contenido
</div>
```

⚠️ **Notar:**

- Doble llave: `{{ }}`
- Propiedades en **camelCase**: `backgroundColor`, `fontSize`
- Valores string entre comillas: `'red'`, `'20px'`
- Valores numéricos sin comillas: `20` (React asume px)

Ejemplos:

```
function Boton() {
  const estiloBoton = {
    backgroundColor: '#007bff',
    color: 'white',
    padding: '10px 20px',
    border: 'none',
    borderRadius: '5px',
    fontSize: '16px',
    cursor: 'pointer'
  }

  return (
    <button style={estiloBoton}>
      Click aquí
    </button>
  )
}
```

O directamente inline:

```
<button style={{
```

```
  backgroundColor: '#007bff',
  color: 'white',
  padding: '10px 20px'
}}>
  Click aquí
</button>
```

Ventajas:

- Rápido para estilos dinámicos
- No hay conflictos de nombres

Desventajas:

- Difícil de mantener en componentes grandes
- No soporta pseudo-clases (:hover, :active)
- No soporta media queries

2.3 CSS Externo (20 min)

La forma más común y recomendada.

PASO 1: Crear un archivo CSS para el componente

Crear `src/components/Header.css`:

```
/* Header.css */
.header {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  padding: 30px;
  text-align: center;
  border-radius: 10px;
  margin-bottom: 20px;
}

.header h1 {
  margin: 0;
  font-size: 2.5rem;
}

.header p {
  margin: 10px 0 0 0;
  font-size: 1.1rem;
  opacity: 0.9;
```

```
}
```

PASO 2: Importar el CSS en el componente

```
// Header.jsx
import './Header.css'  // 👈 Importar CSS

function Header() {
  return (
    <header className="header">  {/* 👈 usar className */}
      <h1>🚀 Mi Aplicación React</h1>
      <p>Bienvenido a mi primera app</p>
    </header>
  )
}

export default Header
```

Guardar y ver los cambios en el navegador.

Ejercicio: Estilizar ProductCard

Crear `src/components/ProductCard.css`:

```
/* ProductCard.css */
.product-card {
  border: 2px solid #e0e0e0;
  padding: 20px;
  margin: 10px;
  border-radius: 12px;
  width: 250px;
  background-color: white;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
  transition: transform 0.2s, box-shadow 0.2s;
}

.product-card:hover {
  transform: translateY(-5px);
  box-shadow: 0 8px 12px rgba(0, 0, 0, 0.15);
}

.product-card img {
  width: 100%;
  height: 200px;
  object-fit: cover;
```

```
  border-radius: 8px;
  margin-bottom: 15px;
}

.product-card h3 {
  margin: 10px 0;
  color: #333;
  font-size: 1.3rem;
}

.product-price {
  font-size: 24px;
  color: #28a745;
  font-weight: bold;
  margin: 10px 0;
}

.product-status {
  font-weight: 600;
  margin-top: 10px;
}

.in-stock {
  color: #28a745;
}

.out-of-stock {
  color: #dc3545;
}
```

Actualizar `ProductCard.jsx`:

```
import './ProductCard.css'

function ProductCard({ nombre, precio, imagen, disponible }) {
  return (
    <div className="product-card">
      <img src={imagen} alt={nombre} />
      <h3>{nombre}</h3>
      <p className="product-price">${precio}</p>
      <p className={disponible ? 'product-status in-stock' : 'product-status out-of-stock'}>
        {disponible ? '✅ En stock' : '❌ Agotado'}
      </p>
    </div>
  )
}
```

export default ProductCard

Notar:

- Clases CSS dinámicas con operador ternario
- Múltiples clases separadas por espacio
- `:hover` funciona perfectamente

2.4 CSS Modules (15 min)

CSS Modules: CSS con scope local (evita conflictos de nombres)

¿Qué problema resuelven?

Si tienes dos componentes con la clase `.titulo`, se pisarán entre sí. CSS Modules previene esto.

PASO 1: Crear archivo con extensión `.module.css`

Crear `src/components/Footer.module.css`:

```
/* Footer.module.css */
.footer {
  background-color: #2c3e50;
  color: white;
  padding: 30px;
  text-align: center;
  margin-top: 40px;
  border-radius: 10px;
}

.titulo {
  font-size: 1.5rem;
  margin-bottom: 10px;
}

.redes {
  display: flex;
  justify-content: center;
  gap: 20px;
  margin-top: 15px;
}

.enlace {
```

```
  color: #3498db;
  text-decoration: none;
  font-weight: 600;
  transition: color 0.3s;
}

.enlace:hover {
  color: #5dade2;
}
```

PASO 2: Importar como objeto y usar

Crear `src/components/Footer.jsx`:

```
import styles from './Footer.module.css'  // 👈 Importar como 'styles'

function Footer() {
  return (
    <footer className={styles.footer}>  {/* 👈 styles.nombreClase */}
      <h3 className={styles.titulo}>Síguenos en Redes</h3>
      <div className={styles.redes}>
        <a href="#" className={styles.enlace}>Twitter</a>
        <a href="#" className={styles.enlace}>Instagram</a>
        <a href="#" className={styles.enlace}>LinkedIn</a>
      </div>
      <p>© 2025 Mi App React</p>
    </footer>
  )
}

export default Footer
```

Usar en `App.jsx`:

```
import Header from './components/Header'
import ProductCard from './components/ProductCard'
import Footer from './components/Footer'

function App() {
  return (
    <div>
      <Header />

      <div style={{ display: 'flex', flexWrap: 'wrap', justifyContent: 'center' }}>
```

```
      <ProductCard
        nombre="Laptop HP"
        precio={1200}
        imagen="<https://picsum.photos/250/200>"
        disponible={true}
      />
      <ProductCard
        nombre="Mouse Logitech"
        precio={25}
        imagen="<https://picsum.photos/250/201>"
        disponible={false}
      />
      <ProductCard
        nombre="Teclado Mecánico"
        precio={80}
        imagen="<https://picsum.photos/250/202>"
        disponible={true}
      />
    </div>

    <Footer />
  </div>
 )
}

export default App
```

Verificar en el navegador: las clases tendrán nombres únicos generados automáticamente

Ejemplo: `.Footer_footer__a8b3c` (previene conflictos)

Comparación de las 3 formas:

Método	Ventajas	Desventajas	Cuándo usar
Inline	Rápido, dinámico	Sin :hover, difícil de mantener	Estilos dinámicos simples
CSS externo	Familiar, completo	Posibles conflictos de nombres	Proyectos pequeños-medianos
CSS Modules	Sin conflictos, escalable	Sintaxis diferente	Proyectos medianos-grandes

---

2.5 Proyecto Guiado Final: Landing Page

Objetivo: Crear una landing page simple integrando todo lo aprendido.

Estructura:

- Header (ya lo tenemos)
- Sección Hero
- Grid de productos (ya lo tenemos)
- Footer (ya lo tenemos)

Crear `src/components/Hero.jsx`:

```
import './Hero.css'

function Hero({ titulo, subtitulo, textoBoton }) {
  return (
    <section className="hero">
      <h1>{titulo}</h1>
      <p>{subtitulo}</p>
      <button className="hero-button">{textoBoton}</button>
    </section>
  )
}

export default Hero
```

Crear `src/components/Hero.css`:

```
/* Hero.css */
.hero {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  padding: 80px 20px;
  text-align: center;
  border-radius: 15px;
  margin-bottom: 40px;
}

.hero h1 {
  font-size: 3rem;
  margin: 0 0 20px 0;
  font-weight: 800;
}
```

```
.hero p {
  font-size: 1.3rem;
  margin-bottom: 30px;
  opacity: 0.95;
  max-width: 600px;
  margin-left: auto;
  margin-right: auto;
}

.hero-button {
  background-color: white;
  color: #667eea;
  padding: 15px 40px;
  font-size: 1.1rem;
  font-weight: 600;
  border: none;
  border-radius: 30px;
  cursor: pointer;
  transition: transform 0.2s, box-shadow 0.2s;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.2);
}

.hero-button:hover {
  transform: translateY(-3px);
  box-shadow: 0 6px 20px rgba(0, 0, 0, 0.3);
}

.hero-button:active {
  transform: translateY(-1px);
}
```

Crear `src/App.css` (estilos del contenedor principal):

```
/* App.css */
.app-container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 20px;
  background-color: #f8f9fa;
  min-height: 100vh;
}

.products-section {
  margin: 40px 0;
```

```
}

.products-title {
  text-align: center;
  font-size: 2.5rem;
  color: #2c3e50;
  margin-bottom: 30px;
}

.products-grid {
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
  gap: 20px;
}
```

Actualizar `src/App.jsx` (versión final integrada):

```
import './App.css'
import Header from './components/Header'
import Hero from './components/Hero'
import ProductCard from './components/ProductCard'
import Footer from './components/Footer'

function App() {
  return (
    <div className="app-container">
      <Header />

      <Hero
        titulo="Bienvenido a TechStore"
        subtitulo="Los mejores productos de tecnología al mejor precio"
        textoBoton="Ver Catálogo"
      />

      <section className="products-section">
        <h2 className="products-title">Nuestros Productos</h2>

        <div className="products-grid">
          <ProductCard
            nombre="Laptop HP Pavilion"
            precio={1200}
            imagen="<https://picsum.photos/250/200?random=1>"
            disponible={true}
          />
```

```
        <ProductCard
          nombre="Mouse Logitech MX"
          precio={25}
          imagen="<https://picsum.photos/250/200?random=2>"
          disponible={false}
        />

        <ProductCard
          nombre="Teclado Mecánico"
          precio={80}
          imagen="<https://picsum.photos/250/200?random=3>"
          disponible={true}
        />

        <ProductCard
          nombre="Monitor Samsung 27"
          precio={350}
          imagen="<https://picsum.photos/250/200?random=4>"
          disponible={true}
        />

        <ProductCard
          nombre="Webcam Logitech 4K"
          precio={120}
          imagen="<https://picsum.photos/250/200?random=5>"
          disponible={true}
        />

        <ProductCard
          nombre="Auriculares Sony"
          precio={200}
          imagen="<https://picsum.photos/250/200?random=6>"
          disponible={false}
        />
      </div>
    </section>

    <Footer />
  </div>
 )
}

export default App
```

Actualizar `src/index.css` (estilos globales):

```
/* index.css */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background-color: #ecf0f1;
  color: #2c3e50;
  line-height: 1.6;
}

h1, h2, h3, h4, h5, h6 {
  font-weight: 700;
}

button {
  font-family: inherit;
}
```

Sesión 3

CLASE (2h): Hooks (useState, useEffect) + React Router

0) Espacio inicial – Discusión de proyecto (20 min)

1. **Estado actual (5 min)**
   - ¿Fase? ¿Entregables cerrados? ¿Bloqueos?
   - Pizarrón: “Hecho / En progreso / Pendiente”.
2. **Desafíos y soluciones (8 min)**
   - Pide 2 bloqueos técnicos por equipo.
   - “¿Se resuelve con estado local? ¿Con efecto? ¿Con rutas? ¿Con levantar estado?”
3. **Próximos pasos (5 min)**
   - Definir 3 tareas por equipo → prioridad alta/media.

    - Asignar responsables y fecha.
4. **Conexión con el tema (2 min)**
    - ¿Qué parte del proyecto **requiere estado**? (formularios, toggles, filtros).
    - ¿Qué parte **requiere efectos**? (fetch, timers, subscripciones).
    - ¿Necesitan **routing**? (múltiples pantallas, detalle por id, 404).

HORA 1 — Hooks: useState y useEffect (60 min)

1.1 Introducción rápida a Hooks (5 min)

- **Qué son**: funciones que agregan capacidades a componentes funcionales.
- **Reglas**:
    - ✅ Llamar **siempre** en el **nivel superior** del componente.
    - ✅ Solo en **componentes React** o **custom hooks**.
    - ❌ No dentro de `if`, `for`, funciones internas.
- Hoy: **useState** (estado local) y **useEffect** (efectos secundarios).

> Nota didáctica: si te descubres diciendo "esto corre después de renderizar", probablemente es un useEffect.

1.2 useState a fondo - Estado en Componentes (25 min)

A) Conceptos clave (3 min)

- **Estado = datos que cambian con el tiempo** → al cambiar, React **re-renderiza**.
- **Firma**: `const [valor, setValor] = useState(estadoInicial)`
    - `variable`: el valor actual del estado
    - `setVariable`: función para actualizar el estado
    - `valorInicial`: valor inicial del estado
- **Inmutabilidad**: nunca modifiques el estado **directamente**.

B) Patrón de actualización funcional (evita "stale state") (4 min)

- Cuando el nuevo valor depende del anterior, **usa función**:

```
const [count, setCount] = useState(0);

// ✅ Correcto: usa el valor previo confiable
setCount(prev => prev + 1);

// ❌ Incorrecto: puede leer un count "viejo" en callbacks asíncronos
setCount(count + 1);
```

Explica “stale closure”: si capturas count en una función y lo usas luego (timer, promesa), puede estar desactualizado. La forma funcional lo evita.

Ejemplo 1 — Contador documentado (5 min)

Crear `src/components/Contador.jsx`:

```
import { useState } from 'react'
import './Contador.css'

function Contador() {
  // Declarar estado
  const [contador, setContador] = useState(0)

  // Funciones para modificar el estado
  const incrementar = () => {
    setContador(contador + 1)
  }

  const decrementar = () => {
    setContador(contador - 1)
  }

  const reiniciar = () => {
    setContador(0)
  }

  return (
    <div className="contador">
      <h2>Contador: {contador}</h2>
      <div className="botones">
        <button onClick={decrementar}>-</button>
        <button onClick={reiniciar}>Reset</button>
        <button onClick={incrementar}>+</button>
      </div>
    </div>
  )
}

export default Contador
```

Crear `src/components/Contador.css`:

```
.contador {
  text-align: center;
```

```
  padding: 30px;
  background-color: white;
  border-radius: 12px;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
  max-width: 400px;
  margin: 20px auto;
}

.contador h2 {
  font-size: 2.5rem;
  color: #667eea;
  margin-bottom: 20px;
}

.botones {
  display: flex;
  gap: 15px;
  justify-content: center;
}

.botones button {
  padding: 12px 30px;
  font-size: 1.2rem;
  font-weight: 600;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  background-color: #667eea;
  color: white;
  transition: all 0.3s;
}

.botones button:hover {
  background-color: #5568d3;
  transform: translateY(-2px);
}
```

Usar en App.jsx:

```
import Contador from './components/Contador'

function App() {
  return (
    <div style={{ padding: '20px' }}>
      <h1 style={{ textAlign: 'center' }}>Demo de useState</h1>
      <Contador />
    </div>
```

```
  )
}

export default App
```

Ejemplo 2 — Toggle con booleano (3 min)

Crear `src/components/Toggle.jsx`:

```
import { useState } from 'react'
import './Toggle.css'

function Toggle() {
  const [visible, setVisible] = useState(true)

  const toggleVisibilidad = () => {
    setVisible(!visible)  // Invierte el valor booleano
  }

  return (
    <div className="toggle-container">
      <button onClick={toggleVisibilidad}>
        {visible ? 'Ocultar' : 'Mostrar'} Contenido
      </button>

      {visible && (
        <div className="contenido">
          <h3>¡Contenido Visible! 👀</h3>
          <p>Este contenido se muestra y oculta con useState</p>
        </div>
      )}
    </div>
  )
}

export default Toggle
```

Crear `src/components/Toggle.css`:

```
.toggle-container {
  padding: 30px;
  text-align: center;
  max-width: 500px;
  margin: 20px auto;
}
```

```
.toggle-container button {
  padding: 12px 30px;
  font-size: 1rem;
  font-weight: 600;
  background-color: #28a745;
  color: white;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  transition: all 0.3s;
}

.toggle-container button:hover {
  background-color: #218838;
}

.contenido {
  margin-top: 20px;
  padding: 20px;
  background-color: #e7f3ff;
  border-radius: 8px;
  border: 2px solid #667eea;
}

.contenido h3 {
  color: #667eea;
  margin-bottom: 10px;
}
```

Ejemplo 3 — Input controlado (form) (5 min)

Crear `src/components/FormularioNombre.jsx`:

```
import { useState } from 'react'
import './FormularioNombre.css'

function FormularioNombre() {
  const [nombre, setNombre] = useState('')
  // Hook useState: crea un estado 'nombre' inicializado como string vacío

  const manejarCambio = (e) => {
    // Define función manejadora de eventos que recibe el evento 'e'
    setNombre(e.target.value)
    // Actualiza el estado 'nombre' con el valor actual del input
    // e.target.value contiene el texto escrito en el input
  }
```

```
  return (
    <div className="formulario-nombre">
      <h2>Hola, {nombre || 'Invitado'}! 👋</h2>

      <input
        type="text"
        placeholder="Escribe tu nombre..."
        value={nombre}
        onChange={manejarCambio}
      />

      <p className="info">Caracteres: {nombre.length}</p>
    </div>
  )
}

export default FormularioNombre
```

Crear `src/components/FormularioNombre.css`:

```
.formulario-nombre {
  text-align: center;
  padding: 30px;
  background-color: white;
  border-radius: 12px;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
  max-width: 500px;
  margin: 20px auto;
}

.formulario-nombre h2 {
  color: #667eea;
  margin-bottom: 20px;
}

.formulario-nombre input {
  width: 100%;
  padding: 12px;
  font-size: 1rem;
  border: 2px solid #ddd;
  border-radius: 8px;
  transition: border-color 0.3s;
}

.formulario-nombre input:focus {
  outline: none;
  border-color: #667eea;
```

```
}

.info {
  margin-top: 10px;
  color: #666;
  font-size: 0.9rem;
}
```

G) Antipatrones y buenas prácticas (3 min)

- ❌ Derivar estado de props y **guardarlo** si puedes **calcularlo en render**.
- ❌ Guardar objetos grandes si sólo cambias un campo → normaliza o separa en varios `useState`.
- ✅ **Levantar estado** al ancestro común si dos hermanos deben compartirlo.
- ✅ Guarda **UI state** local (modales, toggles) en cada componente.

1.3 useEffect a fondo - Efectos Secundarios (30 min)

A) Conceptos clave (4 min)

- **Qué es un efecto**: trabajo *después* del render que **interactúa con el "mundo exterior"**

    - Operaciones que afectan cosas fuera del componente
- (fetch, timers, suscripciones, manipular `document.title`, localStorage, etc.).

- **Firma**:

```
useEffect(() => {
  // Efecto: se ejecuta después del render
  return () => {
    // Limpieza (opcional): se ejecuta antes del próximo efecto o al desmontar
  }
}, [dependencias]) // deps controla cuándo corre
```

- **Dependencias**:
    - `[]` vacío: se ejecuta una sola vez (al montar el componente)
    - `[variable]`: se ejecuta cada vez que `variable` cambia
    - (sin deps) → en **cada** render (casi nunca recomendable).

  Nota: En StrictMode (desarrollo), React invoca efectos dos veces en montaje para ayudarte a encontrar efectos no idempotentes. No pasa en producción.

B) Efecto “título del documento” (5 min)

Crear `src/components/CambiadorTitulo.jsx`:

```
import { useState, useEffect } from 'react'
import './CambiadorTitulo.css'

function CambiadorTitulo() {
  const [contador, setContador] = useState(0)

  // useEffect se ejecuta cada vez que contador cambia
  useEffect(() => {
    document.title = `Contador: ${contador}`

    // Función de limpieza (opcional aquí)
    return () => {
      document.title = 'React App'
    }
  }, [contador])  // 👈 Dependencia: se ejecuta cuando contador cambia

  return (
    <div className="cambiador-titulo">
      <h2>Mira el título de la pestaña del navegador 👆</h2>
      <p>Contador: {contador}</p>
      <button onClick={() => setContador(contador + 1)}>
        Incrementar
      </button>
    </div>
  )
}

export default CambiadorTitulo
```

Crear `src/components/CambiadorTitulo.css`:

```
/* CambiadorTitulo.css */
.cambiador-titulo {
  max-width: 400px;
  margin: 2rem auto;
  padding: 2rem;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  border-radius: 15px;
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.2);
  text-align: center;
  color: white;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}
```

```
.cambiador-titulo h2 {
  margin-bottom: 1.5rem;
  font-size: 1.4rem;
  font-weight: 600;
  text-shadow: 0 2px 4px rgba(0, 0, 0, 0.3);
}

.cambiador-titulo p {
  font-size: 1.8rem;
  margin-bottom: 1.5rem;
  font-weight: bold;
  background: rgba(255, 255, 255, 0.1);
  padding: 0.5rem 1rem;
  border-radius: 8px;
  display: inline-block;
  backdrop-filter: blur(10px);
}

.cambiador-titulo button {
  background: rgba(255, 255, 255, 0.2);
  color: white;
  border: 2px solid rgba(255, 255, 255, 0.3);
  padding: 0.8rem 2rem;
  font-size: 1.1rem;
  border-radius: 25px;
  cursor: pointer;
  transition: all 0.3s ease;
  backdrop-filter: blur(10px);
  font-weight: 600;
}

.cambiador-titulo button:hover {
  background: rgba(255, 255, 255, 0.3);
  border-color: rgba(255, 255, 255, 0.5);
  transform: translateY(-2px);
  box-shadow: 0 5px 15px rgba(0, 0, 0, 0.2);
}

.cambiador-titulo button:active {
  transform: translateY(0);
  transition: transform 0.1s ease;
}

/* Efectos de animación para el contador */
@keyframes pulse {
  0% { transform: scale(1); }
  50% { transform: scale(1.05); }
```

```
  100% { transform: scale(1); }
}

.cambiador-titulo p {
  animation: pulse 0.3s ease;
}

/* Responsive */
@media (max-width: 480px) {
  .cambiador-titulo {
    margin: 1rem;
    padding: 1.5rem;
  }

  .cambiador-titulo h2 {
    font-size: 1.2rem;
  }

  .cambiador-titulo p {
    font-size: 1.5rem;
  }
}
```

C) Fetch con limpieza/abort (8 min)

- Problema clásico: **condiciones de carrera** (llegan respuestas en desorden) o actualizas estado tras desmontar.
- Solución: **AbortController** en la limpieza.

```
// src/components/ListaUsuarios.jsx
import { useState, useEffect } from 'react'
import './ListaUsuarios.css'

function ListaUsuarios() {
const [usuarios, setUsuarios] = useState([])
// Estado para almacenar la lista de usuarios (array vacío inicial)

const [cargando, setCargando] = useState(true)
// Estado para controlar si está cargando (true inicialmente)

const [error, setError] = useState(null)
// Estado para almacenar mensajes de error (null inicialmente)

useEffect(() => {
  // Hook useEffect que se ejecuta una vez al montar el componente

  const controller = new AbortController()
```

```
  // Crea un AbortController para poder cancelar la petición fetch

  async function cargar() {
    // Función asíncrona para cargar los datos
    try {
      setCargando(true)    // Activa el estado de carga
      setError(null)       // Limpia cualquier error anterior

      const resp = await fetch('<https://jsonplaceholder.typicode.com/users>', {
        signal: controller.signal  // Asigna la señal de aborto a la petición
      })

      if (!resp.ok) throw new Error(`HTTP ${resp.status}`)
      // Si la respuesta no es exitosa, lanza error

      const data = await resp.json()
      // Convierte la respuesta a JSON

      setUsuarios(data)    // Guarda los usuarios en el estado

    } catch (e) {
      // Manejo de errores
      if (e.name !== 'AbortError') setError(e.message)
      // Solo actualiza el estado de error si NO es un error de aborto
      // (ignora cancelaciones intencionales)

    } finally {
      setCargando(false)   // Siempre desactiva el estado de carga
    }
  }

  cargar()  // Ejecuta la función de carga

  // Función de limpieza que se ejecuta al desmontar el componente
  return () => controller.abort()

}, []) // 👈 Array de dependencias vacío = se ejecuta solo al montar

// Renderizado condicional
if (cargando) return <div className="lista-usuarios">Cargando...</div>
// Si está cargando, muestra mensaje de carga

if (error) return <div className="lista-usuarios error">Error: {error}</div>
// Si hay error, muestra mensaje de error

  return (
    <div className="lista-usuarios">
      <h2>Usuarios</h2>
```

```
      <div className="usuarios-grid">
        {usuarios.map(u => (
          <div key={u.id} className="usuario-card">
            <h3>{u.name}</h3>
            <p>📧 {u.email}</p>
            <p>🏢 {u.company.name}</p>
          </div>
        ))}
      </div>
    </div>
  )
}

export default ListaUsuarios
```

Timer con limpieza (5 min)

Crear `src/components/Temporizador.jsx`:

```
import { useState, useEffect } from 'react'
import './Temporizador.css'

function Temporizador() {
  const [segundos, setSegundos] = useState(0)
  const [activo, setActivo] = useState(false)

  useEffect(() => {
    let intervalo = null

    if (activo) {
      intervalo = setInterval(() => {
        setSegundos(seg => seg + 1)
      }, 1000)
    }

    // 👇 Función de limpieza: detiene el intervalo
    return () => {
      if (intervalo) {
        clearInterval(intervalo)
      }
    }
  }, [activo])  // Se ejecuta cuando 'activo' cambia

  const toggleTimer = () => {
    setActivo(!activo)
  }
```

```
  const reiniciar = () => {
    setSegundos(0)
    setActivo(false)
  }

  return (
    <div className="temporizador">
      <h2>⏱️ Temporizador</h2>
      <div className="tiempo">{segundos}s</div>
      <div className="controles">
        <button onClick={toggleTimer}>
          {activo ? 'Pausar' : 'Iniciar'}
        </button>
        <button onClick={reiniciar}>Reiniciar</button>
      </div>
    </div>
  )
}

export default Temporizador
```

Crear `src/components/Temporizador.css`:

```
.temporizador {
  text-align: center;
  padding: 40px;
  background-color: white;
  border-radius: 12px;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
  max-width: 400px;
  margin: 20px auto;
}

.temporizador h2 {
  color: #2c3e50;
  margin-bottom: 20px;
}

.tiempo {
  font-size: 4rem;
  font-weight: 700;
  color: #667eea;
  margin: 30px 0;
}

.controles {
  display: flex;
```

```
  gap: 15px;
  justify-content: center;
}

.controles button {
  padding: 12px 30px;
  font-size: 1rem;
  font-weight: 600;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  background-color: #667eea;
  color: white;
  transition: all 0.3s;
}

.controles button:hover {
  background-color: #5568d3;
}
```

HORA 2 — React Router v6 (60 min)

2.1 ¿Qué es React Router? (5 min)

React Router es una librería para manejar navegación en aplicaciones React.

¿Por qué lo necesitamos?

- React es SPA (Single Page Application)
- Necesitamos múltiples "páginas" sin recargar
- Queremos URLs limpias y navegación fluida
- Necesitamos navegación programática

Instalación:

```
npm install react-router-dom
```

2.2 Conceptos Básicos (10 min)

Componentes principales:

1. `<BrowserRouter>`: Envuelve toda la app
2. `<Routes>`: Contenedor de rutas
3. `<Route>`: Define una ruta

4. `<Link>`: Navegación sin recargar
5. `useNavigate`: Navegación programática

Ejemplo simple de estructura:

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
  </Routes>
</BrowserRouter>
```

2.3 Crear Páginas (20 min)

Crear carpeta `src/pages/`

Crear `src/pages/Home.jsx`:

```
import './Page.css'

function Home() {
  return (
    <div className="page home-page">
      <h1>🏠 Bienvenido a Mi App</h1>
      <p className="intro">
        Esta es una aplicación de demostración de React con Hooks y React Router.
      </p>

      <div className="features">
        <div className="feature-card">
          <h3>⚡ Rápido</h3>
          <p>Navegación sin recargas de página</p>
        </div>
        <div className="feature-card">
          <h3>🎨 Moderno</h3>
          <p>Diseño limpio y responsive</p>
        </div>
        <div className="feature-card">
          <h3>🚀 Potente</h3>
          <p>Construido con React y Hooks</p>
        </div>
      </div>
    </div>
  )
}
```

```
export default Home
```

Crear `src/pages/Home.css`:

```
.home-page {
  text-align: center;
}

.intro {
  font-size: 1.2rem;
  color: #666;
  max-width: 600px;
  margin: 20px auto 50px;
}

.features {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 30px;
  max-width: 900px;
  margin: 0 auto;
}

.feature-card {
  background-color: white;
  padding: 40px 30px;
  border-radius: 12px;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
  transition: transform 0.3s;
}

.feature-card:hover {
  transform: translateY(-10px);
  box-shadow: 0 8px 12px rgba(0, 0, 0, 0.15);
}

.feature-card h3 {
  color: #667eea;
  font-size: 1.5rem;
  margin-bottom: 15px;
}

.feature-card p {
  color: #666;
  font-size: 1rem;
}
```

Crear `src/pages/About.jsx`:

```
import './Page.css'

function About() {
  return (
    <div className="page about-page">
      <h1>📖 Sobre Nosotros</h1>

      <div className="about-content">
        <section className="about-section">
          <h2>Nuestra Historia</h2>
          <p>
            Somos un equipo apasionado por crear experiencias web increíbles
            usando las últimas tecnologías de React.
          </p>
        </section>

        <section className="about-section">
          <h2>Nuestra Misión</h2>
          <p>
            Proporcionar aplicaciones web rápidas, accesibles y hermosas
            que mejoren la vida de nuestros usuarios.
          </p>
        </section>

        <section className="about-section">
          <h2>Tecnologías</h2>
          <div className="tech-list">
            <span className="tech-badge">⚛️ React</span>
            <span className="tech-badge">🎣 Hooks</span>
            <span className="tech-badge">🛣️ React Router</span>
            <span className="tech-badge">🎨 CSS3</span>
          </div>
        </section>
      </div>
    </div>
  )
}

export default About
```

Crear `src/pages/About.css`:

```
.about-page {
```

```
  max-width: 800px;
  margin: 0 auto;
}

.about-content {
  background-color: white;
  padding: 40px;
  border-radius: 12px;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}

.about-section {
  margin-bottom: 40px;
}

.about-section:last-child {
  margin-bottom: 0;
}

.about-section h2 {
  color: #667eea;
  margin-bottom: 15px;
}

.about-section p {
  color: #555;
  line-height: 1.6;
  font-size: 1.1rem;
}

.tech-list {
  display: flex;
  flex-wrap: wrap;
  gap: 15px;
  margin-top: 20px;
}

.tech-badge {
  background-color: #667eea;
  color: white;
  padding: 10px 20px;
  border-radius: 20px;
  font-weight: 600;
}
```

Crear `src/pages/Contact.jsx`:

```
import { useState } from 'react'
import './Contact.css'

function Contact() {
  const [formData, setFormData] = useState({
    nombre: '',
    email: '',
    mensaje: ''
  })

  const [enviado, setEnviado] = useState(false)

  const manejarCambio = (e) => {
    setFormData({
      ...formData,
      [e.target.name]: e.target.value
    })
  }

  const manejarEnvio = (e) => {
    e.preventDefault()
    console.log('Datos del formulario:', formData)
    setEnviado(true)

    // Reiniciar después de 3 segundos
    setTimeout(() => {
      setEnviado(false)
      setFormData({ nombre: '', email: '', mensaje: '' })
    }, 3000)
  }

  return (
    <div className="page contact-page">
      <h1>📬 Contáctanos</h1>

      {enviado ? (
        <div className="mensaje-exito">
          <h2>✅ ¡Mensaje enviado con éxito!</h2>
          <p>Nos pondremos en contacto pronto.</p>
        </div>
      ) : (
        <form className="contact-form" onSubmit={manejarEnvio}>
          <div className="form-group">
            <label>Nombre:</label>
            <input
              type="text"
              name="nombre"
              value={formData.nombre}
```

```
            onChange={manejarCambio}
            required
          />
        </div>

        <div className="form-group">
          <label>Email:</label>
          <input
            type="email"
            name="email"
            value={formData.email}
            onChange={manejarCambio}
            required
          />
        </div>

        <div className="form-group">
          <label>Mensaje:</label>
          <textarea
            name="mensaje"
            value={formData.mensaje}
            onChange={manejarCambio}
            rows="5"
            required
          />
        </div>

        <button type="submit">Enviar Mensaje</button>
      </form>
    )}
  </div>
)
}

export default Contact
```

Crear `src/pages/Contact.css`:

```
.contact-page {
  max-width: 600px;
  margin: 0 auto;
}

.contact-form {
  background-color: white;
  padding: 40px;
  border-radius: 12px;
```

```
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}

.form-group {
  margin-bottom: 20px;
}

.form-group label {
  display: block;
  margin-bottom: 8px;
  color: #333;
  font-weight: 600;
}

.form-group input,
.form-group textarea {
  width: 100%;
  padding: 12px;
  border: 2px solid #ddd;
  border-radius: 8px;
  font-size: 1rem;
  font-family: inherit;
  transition: border-color 0.3s;
}

.form-group input:focus,
.form-group textarea:focus {
  outline: none;
  border-color: #667eea;
}

.contact-form button {
  width: 100%;
  padding: 15px;
  background-color: #667eea;
  color: white;
  border: none;
  border-radius: 8px;
  font-size: 1.1rem;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.3s;
}

.contact-form button:hover {
  background-color: #5568d3;
}
```

```
.mensaje-exito {
  background-color: white;
  padding: 60px 40px;
  border-radius: 12px;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
  text-align: center;
}

.mensaje-exito h2 {
  color: #28a745;
  margin-bottom: 15px;
}
```

Crear estilos globales para todas las páginas `src/pages/Page.css`:

```
.page {
  padding: 40px 20px;
  min-height: calc(100vh - 200px);
}

.page h1 {
  color: #2c3e50;
  margin-bottom: 30px;
  font-size: 2.5rem;
}
```

Importar en cada página:

```
import './Page.css'
```

2.4 Configurar React Router (15 min)

Actualizar `src/App.jsx`:

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom'
import Home from './pages/Home'
import About from './pages/About'
import Contact from './pages/Contact'
import './App.css'

function App() {
  return (
    <BrowserRouter>
```

```
      <div className="app">
        {/* Navbar de navegación */}
        <nav className="navbar">
          <div className="nav-brand">Mi App</div>
          <div className="nav-links">
            <Link to="/">Inicio</Link>
            <Link to="/about">Nosotros</Link>
            <Link to="/contact">Contacto</Link>
          </div>
        </nav>

        {/* Contenido que cambia según la ruta */}
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/about" element={<About />} />
          <Route path="/contact" element={<Contact />} />
        </Routes>

        {/* Footer */}
        <footer className="footer">
          <p>© 2025 Mi App React - Todos los derechos reservados</p>
        </footer>
      </div>
    </BrowserRouter>
  )
}

export default App
```

Actualizar `src/App.css`:

```
.app {
  min-height: 100vh;
  background-color: #f8f9fa;
}

/* Navbar */
.navbar {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  padding: 20px 40px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
}

.nav-brand {
```

```
  color: white;
  font-size: 1.5rem;
  font-weight: 700;
}

.nav-links {
  display: flex;
  gap: 30px;
}

.nav-links a {
  color: white;
  text-decoration: none;
  font-weight: 600;
  font-size: 1.1rem;
  transition: opacity 0.3s;
}

.nav-links a:hover {
  opacity: 0.8;
}

/* Footer */
.footer {
  background-color: #2c3e50;
  color: white;
  text-align: center;
  padding: 30px;
  margin-top: 60px;
}

.footer p {
  margin: 0;
}
```

Explicar los componentes de React Router:

1. **`<BrowserRouter>`**: Envuelve toda la app para habilitar el routing
2. **`<Routes>`**: Contenedor para todas las rutas
3. **`<Route>`**: Define una ruta específica
   - `path`: la URL (ej: "/about")
   - `element`: el componente a mostrar
4. **`<Link>`**: Enlace para navegar (reemplaza `<a>`)
   - `to`: la ruta de destino

¡Guardar y probar en el navegador!

Ahora deberían poder navegar entre páginas sin recargar.

2.5 Navegación Programática (10 min)

A veces necesitamos navegar mediante código (no con clicks)

Ejemplo: Redirigir después de enviar un formulario

Actualizar `src/pages/Contact.jsx`:

```
import { useState } from 'react'
import { useNavigate } from 'react-router-dom'  // 👈 Importar hook
import './Contact.css'

function Contact() {
  const navigate = useNavigate()  // 👈 Obtener función de navegación
  const [formData, setFormData] = useState({
    nombre: '',
    email: '',
    mensaje: ''
  })

  const manejarCambio = (e) => {
    setFormData({
      ...formData,
      [e.target.name]: e.target.value
    })
  }

  const manejarEnvio = (e) => {
    e.preventDefault()
    console.log('Datos del formulario:', formData)

    // Simular envío
    alert('¡Mensaje enviado! Redirigiendo a inicio...')

    // Navegar programáticamente
    navigate('/')  // 👈 Redirige a Home
  }

  return (
    <div className="page contact-page">
      <h1>📬 Contáctanos</h1>

      <form className="contact-form" onSubmit={manejarEnvio}>
```

```
      <div className="form-group">
        <label>Nombre:</label>
        <input
          type="text"
          name="nombre"
          value={formData.nombre}
          onChange={manejarCambio}
          required
        />
      </div>

      <div className="form-group">
        <label>Email:</label>
        <input
          type="email"
          name="email"
          value={formData.email}
          onChange={manejarCambio}
          required
        />
      </div>

      <div className="form-group">
        <label>Mensaje:</label>
        <textarea
          name="mensaje"
          value={formData.mensaje}
          onChange={manejarCambio}
          rows="5"
          required
        />
      </div>

      <button type="submit">Enviar Mensaje</button>
    </form>
  </div>
 )
}

export default Contact
```

Hook `useNavigate`:

- `navigate('/')`: ir a una ruta
- `navigate(-1)`: ir atrás
- `navigate(1)`: ir adelante

CLASE: CRUD Simple con MockAPI (2 horas)

PREPARACIÓN: Configurar MockAPI (10 min)

mongodb+srv://jacobogarcesoquendo:aFJzVMGN3o7fA38A@cluster0.mqwbn.mongodb.net/{nombre} Esta es la base de datos

Crear cuenta y proyecto en MockAPI

1. Ir a: https://mockapi.io

2. Crear cuenta (con email o GitHub)

3. Crear proyecto:

- Click en "**+ New Project**"
- Nombre: "productos"

4. Crear recurso:

- Click en "**+ New Resource**"
- Nombre: `products`

5. Agregar campos:

Campo	Tipo
name	string
price	number
category	string

6. Click en "Generate Data" para crear productos de ejemplo

7. Copiar tu URL:

https://[tu-id].mockapi.io/api/v1/products

HORA 1: Listar y Crear Productos (60 min)

1.1 Crear estructura básica (5 min)

```
src/
├── App.jsx
├── App.css
└── main.jsx
```

Vamos a hacer TODO en un solo archivo: `App.jsx`

1.2 Configuración inicial de App.jsx (10 min)

```
import { useState, useEffect } from 'react'
import './App.css'

// 👇 REEMPLAZA CON TU URL DE MOCKAPI
const API_URL = 'https://[TU-ID].mockapi.io/api/v1/products'

function App() {
  // Estados
  const [productos, setProductos] = useState([])
  const [nombre, setNombre] = useState('')
  const [precio, setPrecio] = useState('')
  const [categoria, setCategoria] = useState('')

  return (
    <div className="app">
      <h1>🛒 CRUD de Productos</h1>
    </div>
  )
}

export default App
```

1.3 Operación READ - Obtener productos (15 min)

Agregar useEffect para cargar productos:

```
function App() {
  const [productos, setProductos] = useState([])
  const [nombre, setNombre] = useState('')
```

```
  const [precio, setPrecio] = useState('')
  const [categoria, setCategoria] = useState('')

  // 👇 Cargar productos al iniciar
  useEffect(() => {
    fetch(API_URL)
      .then(res => res.json())
      .then(data => setProductos(data))
  }, [])

  return (
    <div className="app">
      <h1>🛒 CRUD de Productos</h1>

      {/* Mostrar productos */}
      <div className="lista">
        <h2>Lista de Productos</h2>
        {productos.map(producto => (
          <div key={producto.id} className="producto">
            <h3>{producto.name}</h3>
            <p>Precio: ${producto.price}</p>
            <p>Categoría: {producto.category}</p>
          </div>
        ))}
      </div>
    </div>
  )
}
```

Guardar y verificar que se muestren los productos.

1.4 Operación CREATE - Crear producto (30 min)

Agregar formulario y función para crear:

```
function App() {
  const [productos, setProductos] = useState([])
  const [nombre, setNombre] = useState('')
  const [precio, setPrecio] = useState('')
  const [categoria, setCategoria] = useState('')

  // Cargar productos
  useEffect(() => {
    fetch(API_URL)
```

```
    .then(res => res.json())
    .then(data => setProductos(data))
}, [])

// 👇 Función para crear producto
const crearProducto = () => {
  const nuevoProducto = {
    name: nombre,
    price: precio,
    category: categoria
  }

  fetch(API_URL, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(nuevoProducto)
  })
    .then(res => res.json())
    .then(data => {
      setProductos([...productos, data])  // Agregar a la lista
      // Limpiar formulario
      setNombre('')
      setPrecio('')
      setCategoria('')
    })
}

return (
  <div className="app">
    <h1>🛒 CRUD de Productos</h1>

    {/* Formulario para crear */}
    <div className="formulario">
      <h2>Crear Producto</h2>
      <input
        type="text"
        placeholder="Nombre"
        value={nombre}
        onChange={(e) => setNombre(e.target.value)}
      />
      <input
        type="number"
        placeholder="Precio"
        value={precio}
        onChange={(e) => setPrecio(e.target.value)}
      />
      <input
        type="text"
```

```
        placeholder="Categoría"
        value={categoria}
        onChange={(e) => setCategoria(e.target.value)}
      />
      <button onClick={crearProducto}>Crear</button>
    </div>

    {/* Lista de productos */}
    <div className="lista">
      <h2>Lista de Productos</h2>
      {productos.map(producto => (
        <div key={producto.id} className="producto">
          <h3>{producto.name}</h3>
          <p>Precio: ${producto.price}</p>
          <p>Categoría: {producto.category}</p>
        </div>
      ))}
    </div>
  </div>
 )
}
```

Probar creando un producto nuevo.

HORA 2: Actualizar y Eliminar (60 min)

2.1 Operación DELETE - Eliminar producto (20 min)

Agregar función y botón para eliminar:

```
function App() {
  const [productos, setProductos] = useState([])
  const [nombre, setNombre] = useState('')
  const [precio, setPrecio] = useState('')
  const [categoria, setCategoria] = useState('')

  useEffect(() => {
    fetch(API_URL)
      .then(res => res.json())
      .then(data => setProductos(data))
  }, [])

  const crearProducto = () => {
```

```
  const nuevoProducto = {
    name: nombre,
    price: precio,
    category: categoria
  }

  fetch(API_URL, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(nuevoProducto)
  })
    .then(res => res.json())
    .then(data => {
      setProductos([...productos, data])
      setNombre('')
      setPrecio('')
      setCategoria('')
    })
}

// 👇 Función para eliminar
const eliminarProducto = (id) => {
  fetch(`${API_URL}/${id}`, {
    method: 'DELETE'
  })
    .then(() => {
      // Quitar de la lista
      setProductos(productos.filter(p => p.id !== id))
    })
}

return (
  <div className="app">
    <h1>🛒 CRUD de Productos</h1>

    <div className="formulario">
      <h2>Crear Producto</h2>
      <input
        type="text"
        placeholder="Nombre"
        value={nombre}
        onChange={(e) => setNombre(e.target.value)}
      />
      <input
        type="number"
        placeholder="Precio"
        value={precio}
        onChange={(e) => setPrecio(e.target.value)}
```

```
      />
      <input
        type="text"
        placeholder="Categoría"
        value={categoria}
        onChange={(e) => setCategoria(e.target.value)}
      />
      <button onClick={crearProducto}>Crear</button>
    </div>

    <div className="lista">
      <h2>Lista de Productos</h2>
      {productos.map(producto => (
        <div key={producto.id} className="producto">
          <h3>{producto.name}</h3>
          <p>Precio: ${producto.price}</p>
          <p>Categoría: {producto.category}</p>
          {/* 👇 Botón eliminar */}
          <button onClick={() => eliminarProducto(producto.id)}>
            Eliminar
          </button>
        </div>
      ))}
    </div>
  </div>
  )
}
```

Probar eliminando productos.

2.2 Operación UPDATE - Actualizar producto (40 min)

Agregar estado y funciones para editar:

```
function App() {
  const [productos, setProductos] = useState([])
  const [nombre, setNombre] = useState('')
  const [precio, setPrecio] = useState('')
  const [categoria, setCategoria] = useState('')
  const [editando, setEditando] = useState(null)  // 👈 ID del producto que se está editando

  useEffect(() => {
    fetch(API_URL)
      .then(res => res.json())
```

```
    .then(data => setProductos(data))
}, [])

const crearProducto = () => {
  const nuevoProducto = {
    name: nombre,
    price: precio,
    category: categoria
  }

  fetch(API_URL, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(nuevoProducto)
  })
    .then(res => res.json())
    .then(data => {
      setProductos([...productos, data])
      setNombre('')
      setPrecio('')
      setCategoria('')
    })
}

const eliminarProducto = (id) => {
  fetch(`${API_URL}/${id}`, {
    method: 'DELETE'
  })
    .then(() => {
      setProductos(productos.filter(p => p.id !== id))
    })
}

// 👇 Función para preparar edición
const prepararEdicion = (producto) => {
  setEditando(producto.id)
  setNombre(producto.name)
  setPrecio(producto.price)
  setCategoria(producto.category)
}

// 👇 Función para actualizar
const actualizarProducto = () => {
  const productoActualizado = {
    name: nombre,
    price: precio,
    category: categoria
  }
```

```
  fetch(`${API_URL}/${editando}`, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(productoActualizado)
  })
    .then(res => res.json())
    .then(data => {
      // Actualizar en la lista
      setProductos(productos.map(p =>
        p.id === editando ? data : p
      ))
      // Limpiar
      setEditando(null)
      setNombre('')
      setPrecio('')
      setCategoria('')
    })
}

return (
  <div className="app">
    <h1>🛒 CRUD de Productos</h1>

    <div className="formulario">
      <h2>{editando ? 'Editar Producto' : 'Crear Producto'}</h2>
      <input
        type="text"
        placeholder="Nombre"
        value={nombre}
        onChange={(e) => setNombre(e.target.value)}
      />
      <input
        type="number"
        placeholder="Precio"
        value={precio}
        onChange={(e) => setPrecio(e.target.value)}
      />
      <input
        type="text"
        placeholder="Categoría"
        value={categoria}
        onChange={(e) => setCategoria(e.target.value)}
      />

      {/* 👇 Botón condicional */}
      {editando ? (
        <button onClick={actualizarProducto}>Actualizar</button>
```

```
      ) : (
        <button onClick={crearProducto}>Crear</button>
      )}

      {editando && (
        <button onClick={() => {
          setEditando(null)
          setNombre('')
          setPrecio('')
          setCategoria('')
        }}>
          Cancelar
        </button>
      )}
    </div>

    <div className="lista">
      <h2>Lista de Productos</h2>
      {productos.map(producto => (
        <div key={producto.id} className="producto">
          <h3>{producto.name}</h3>
          <p>Precio: ${producto.price}</p>
          <p>Categoría: {producto.category}</p>
          <button onClick={() => prepararEdicion(producto)}>
            Editar
          </button>
          <button onClick={() => eliminarProducto(producto.id)}>
            Eliminar
          </button>
        </div>
      ))}
    </div>
  </div>
  )
}

export default App
```

2.3 Estilos básicos (App.css)

```
.app {
  max-width: 800px;
  margin: 0 auto;
  padding: 20px;
  font-family: Arial, sans-serif;
```

```
}

h1 {
  text-align: center;
  color: #333;
}

.formulario {
  background: #f5f5f5;
  padding: 20px;
  border-radius: 8px;
  margin-bottom: 30px;
}

.formulario h2 {
  margin-top: 0;
}

.formulario input {
  width: 100%;
  padding: 10px;
  margin-bottom: 10px;
  border: 1px solid #ddd;
  border-radius: 4px;
  font-size: 16px;
}

.formulario button {
  padding: 10px 20px;
  margin-right: 10px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  font-size: 16px;
  background-color: #4CAF50;
  color: white;
}

.formulario button:hover {
  background-color: #45a049;
}

.lista h2 {
  border-bottom: 2px solid #333;
  padding-bottom: 10px;
}

.producto {
```

```
  background: white;
  padding: 15px;
  margin-bottom: 15px;
  border: 1px solid #ddd;
  border-radius: 8px;
}

.producto h3 {
  margin: 0 0 10px 0;
  color: #333;
}

.producto p {
  margin: 5px 0;
  color: #666;
}

.producto button {
  padding: 8px 15px;
  margin-right: 10px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  font-size: 14px;
}

.producto button:first-of-type {
  background-color: #ffc107;
  color: black;
}

.producto button:last-of-type {
  background-color: #f44336;
  color: white;
}
```

CÓDIGO COMPLETO FINAL

`src/App.jsx`:

```
import { useState, useEffect } from 'react'
import './App.css'

const API_URL = 'https://[TU-ID].mockapi.io/api/v1/products'
```

```
function App() {
  const [productos, setProductos] = useState([])
  const [nombre, setNombre] = useState('')
  const [precio, setPrecio] = useState('')
  const [categoria, setCategoria] = useState('')
  const [editando, setEditando] = useState(null)

  // READ - Obtener productos
  useEffect(() => {
    fetch(API_URL)
      .then(res => res.json())
      .then(data => setProductos(data))
  }, [])

  // CREATE - Crear producto
  const crearProducto = () => {
    fetch(API_URL, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ name: nombre, price: precio, category: categoria })
    })
      .then(res => res.json())
      .then(data => {
        setProductos([...productos, data])
        limpiarFormulario()
      })
  }

  // UPDATE - Actualizar producto
  const actualizarProducto = () => {
    fetch(`${API_URL}/${editando}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ name: nombre, price: precio, category: categoria })
    })
      .then(res => res.json())
      .then(data => {
        setProductos(productos.map(p => p.id === editando ? data : p))
        limpiarFormulario()
        setEditando(null)
      })
  }

  // DELETE - Eliminar producto
  const eliminarProducto = (id) => {
    fetch(`${API_URL}/${id}`, { method: 'DELETE' })
      .then(() => {
```

```
      setProductos(productos.filter(p => p.id !== id))
    })
  }

  // Preparar edición
  const prepararEdicion = (producto) => {
    setEditando(producto.id)
    setNombre(producto.name)
    setPrecio(producto.price)
    setCategoria(producto.category)
  }

  // Limpiar formulario
  const limpiarFormulario = () => {
    setNombre('')
    setPrecio('')
    setCategoria('')
  }

  return (
    <div className="app">
      <h1>🛒 CRUD de Productos</h1>

      {/* FORMULARIO */}
      <div className="formulario">
        <h2>{editando ? 'Editar Producto' : 'Crear Producto'}</h2>
        <input
          type="text"
          placeholder="Nombre"
          value={nombre}
          onChange={(e) => setNombre(e.target.value)}
        />
        <input
          type="number"
          placeholder="Precio"
          value={precio}
          onChange={(e) => setPrecio(e.target.value)}
        />
        <input
          type="text"
          placeholder="Categoría"
          value={categoria}
          onChange={(e) => setCategoria(e.target.value)}
        />

        {editando ? (
          <>
            <button onClick={actualizarProducto}>Actualizar</button>
```

```
          <button onClick={() => { setEditando(null); limpiarFormulario() }}>
            Cancelar
          </button>
        </>
      ) : (
        <button onClick={crearProducto}>Crear</button>
      )}
    </div>

    {/* LISTA DE PRODUCTOS */}
    <div className="lista">
      <h2>Lista de Productos ({productos.length})</h2>
      {productos.map(producto => (
        <div key={producto.id} className="producto">
          <h3>{producto.name}</h3>
          <p>Precio: ${producto.price}</p>
          <p>Categoría: {producto.category}</p>
          <button onClick={() => prepararEdicion(producto)}>Editar</button>
          <button onClick={() => eliminarProducto(producto.id)}>Eliminar</button>
        </div>
      ))}
    </div>
  </div>
 )
}

export default App
```

RESUMEN DE MÉTODOS HTTP

Operación	Método	Código
CREATE	POST	`fetch(URL, { method: 'POST', ... })`
READ	GET	`fetch(URL)`
UPDATE	PUT	`fetch(URL/id, { method: 'PUT', ... })`
DELETE	DELETE	`fetch(URL/id, { method: 'DELETE' })`

EJERCICIO PARA PRACTICAR

Agregar un campo "stock" al CRUD:

1. Agregar el campo en MockAPI
2. Agregar input en el formulario
3. Agregar estado `const [stock, setStock] = useState('')`
4. Incluir en las peticiones POST y PUT
5. Mostrar en la lista

¡ESO ES TODO!

Con este código tienes un CRUD completo y funcional. Todo en un solo archivo, simple y directo al punto.

Conceptos clave:

- ✅ `useEffect` para cargar datos
- ✅ `useState` para manejar formulario y lista
- ✅ `fetch` para todas las operaciones
- ✅ Métodos HTTP: GET, POST, PUT, DELETE